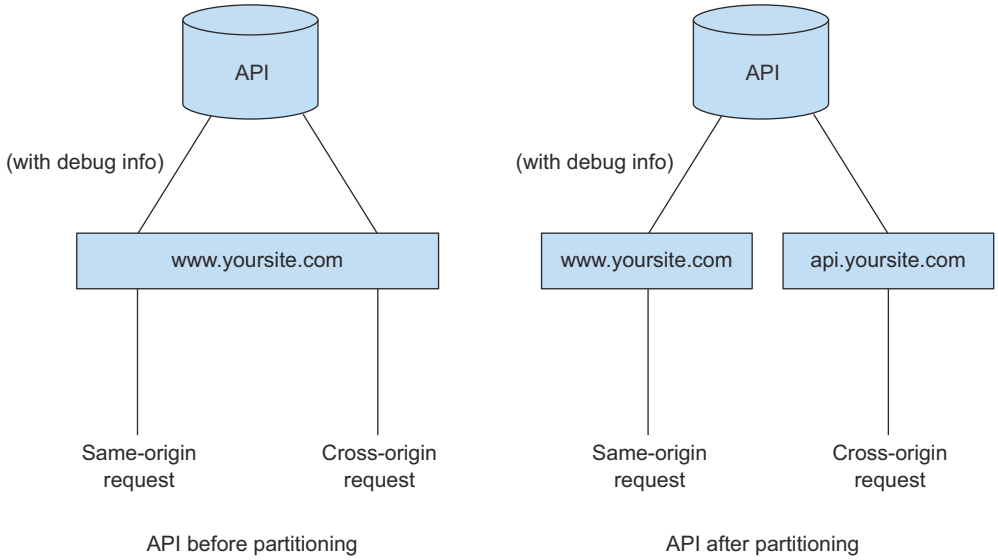


Figure 6.9 Splitting an API into same-origin and cross-origin components

figure out the scheme of the request. Calling `req.protocol` returns either `http` or `https`, and calculating the scheme is as easy as appending :// to the protocol (for example, if the URL's protocol is "http", the scheme is "http://"). This `protocol` property is only available in Express. If you use Node without Express, you may need to find some other way of getting the request's scheme.

A better alternative to the Host header method is to separate same-origin traffic from cross-origin traffic (figure 6.9). If your same-origin and cross-origin traffic all funnels through www.yoursite.com, consider separating API requests into two different servers: www.yoursite.com for same-origin traffic, and http://api.yoursite.com for cross-origin traffic. You don't have to implement two completely different APIs, you just need to channel the requests differently. It's like having two doors to enter a bank: one door is for the public, and the second is for employees only.

Hopefully this section has given you a more comprehensive understanding of how to use the Access-Control-Allow-Origin header to manage cross-origin requests to your API. Although the Access-Control-Allow-Origin header helps dictate who gets access to your server, it shouldn't be the sole security mechanism for your API. The next section looks at the security issues related to CORS.

6.4 Security

The previous section presented strategies for accessing a server from different origins. At one end of the spectrum, any client can make cross-origin requests by setting the `Access-Control-Allow-Origin: *` header. At the other end of the spectrum, a server administrator can limit cross-origin requests to specific clients only. But regardless of which strategy you choose, there is one thing to always keep in mind: CORS isn't security.